

# ROBOCUPJUNIOR RESCUE LINE 2026

## TEAM DESCRIPTION PAPER

### NLO Rescue CRO Team

## Abstract

Our robot was designed and built specifically for the RoboCupJunior Rescue Line 2026 category. We started work on the 3D design and parts testing in September 2025. During development we built two different robots: the first one in December 2025 and the second one in March 2026. The robot is based on a modular architecture using a Teensy 4.1 microcontroller and a Raspberry Pi for vision processing. We use two cameras: one for line tracking and one for the evacuation zone. Both cameras are connected to the Raspberry Pi and programmed in OpenCV, allowing precise image processing. To keep the lighting stable regardless of the environment, we developed a custom diode-based lighting system around the line camera. All low-level control, including motors and sensors, is handled by the Teensy 4.1, while the Raspberry Pi processes visual data and runs a custom neural network for victim detection. The two controllers communicate using the UART protocol. The robot is built around a 3D printed chassis, a custom made PCB and sensors needed for the competition. Through development and testing, including multiple robot and PCB versions, we achieved a reliable and efficient robot capable of handling all challenges of the Rescue Line competition.

Image 1 (first robot)

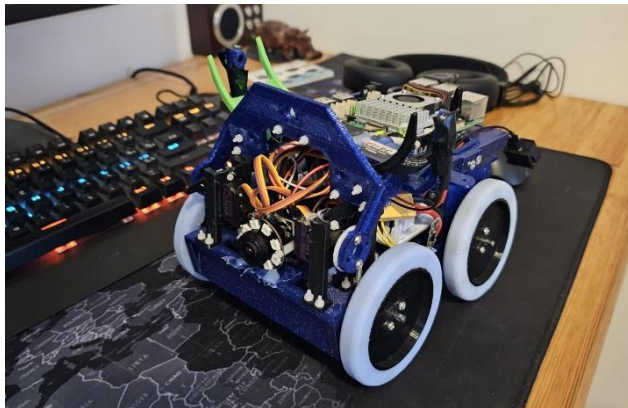
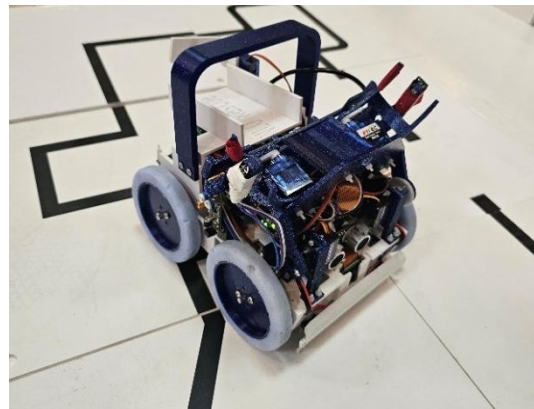


Image 2 (second robot)



## 1. Introduction

### a. Team

**Sven Ridzak – Team captain, software (evacuation zone), PCB design, hardware**

Sven leads the team and is responsible for the software that runs inside the evacuation zone, including victim detection, victim handling and exit search. He also designed and built the custom made PCB and worked on the overall hardware integration. His past experience with PCB design and electronics from previous RoboCup competitions helped us reduce wiring and improve reliability.

### Jan Ridzak – Software (line following), 3D modelling

Jan is responsible for the line following software, including image processing of the line camera, intersection handling and red line detection. He also designs all 3D printed parts of the robot in Fusion 360, including the chassis, wheels and mounts. His experience from past Rescue Line competitions influenced the mechanical design and the modular approach we use.

### Juraj Kolarić and Ivica Kolarić– Team mentors

Juraj and Ivica are the mentors of this team and support the team with advice on engineering decisions, project planning and competition strategy.

## b. About the Team

We are a team from Zagreb, Croatia. We are twins and have been actively participating in RoboCupJunior competitions since 2019, starting in the Speed Line category. Over the years we continuously improved our knowledge in robotics, programming and electronics. We participated in many national and international competitions, including five European RoboCupJunior championships. Each competition allowed us to refine our design process and develop more advanced robots. Our experience from previous years influenced the design of this robot, especially in terms of robustness and software.

## 2. Project Planning

### a. Overall Project Plan

Our primary objective for this competition is to achieve the best possible result and to win. Based on our previous experience in Rescue Line and Rescue Line Entry, we focused on designing a robot that is reliable, fast and capable of solving all elements of the track consistently. We wanted a robot that handles line following, intersections, ramps, obstacles and the evacuation zone without manual intervention. To reach this goal we first wrote down all the rules and challenges of the Rescue Line 2026 competition and turned them into a software and hardware requirement. This way we could check during testing whether our robot really solves every element of the track. The table below shows the main challenges we identified and how we addressed each of them on the software and hardware side.

Table 1. Mapping of competition challenges to software and hardware solutions.

| Challenge                                                                             | Software                                                                                  | Hardware                                                                      |
|---------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Robot must follow a black line on a white surface with gaps, intersections and curves | Line position calculated from camera image using OpenCV; PID control for smooth following | Downward facing line camera with custom diode lighting for stable image       |
| Robot must detect green markers at intersections and turn in the correct direction    | Colour segmentation of camera image to find green markers next to the line                | Same line camera; lighting diodes keep green colour consistent on every track |
| Robot must climb ramps without slipping and pass bumps                                | Ramp detection using IMU angle; motor speed is increased on ramps                         | Custom silicone coated wheels for grip; low centre of gravity chassis         |

|                                                                       |                                                                                      |                                                                                                                                                                |
|-----------------------------------------------------------------------|--------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Robot must avoid obstacles on the line                                | Obstacle avoiding subprogram that drives around the obstacle and returns to the line | Ultrasonic sensor on the front of the robot (later removed and replaced with a better and quicker IR sensor)                                                   |
| Robot must detect the silver line and stop before the evacuation zone | Silver colour detection in line camera image, silver line detection                  | Same line camera; tuned colour thresholds; lidar for detecting the silver line                                                                                 |
| Robot must find live and dead victims in the evacuation zone          | Custom neural network trained in Ultralytics on 2000 labelled images                 | PiCamera mounted on the front of the robot for a wide view of the zone                                                                                         |
| Robot must pick up victims and drop them in the correct area          | Find the drop zones using neural network                                             | Arm with 180° servos for lifting and 360° servos for grabbing; collection area for storing the live victims; photodiode for detecting the victim is in the arm |
| Robot must find the exit of the evacuation zone after rescue          | Wall following subprogram using IMU and IR sensors                                   | Side IR sensors for distance to wall                                                                                                                           |
| Neural network must run fast enough on the Raspberry Pi               | Software optimized inference using the Hailo framework on the Raspberry Pi           | Raspberry Pi with active cooling; Hailo AI accelerator                                                                                                         |

After defining the requirements, we wrote a schedule with provisional milestones. The national competition in March 2026, the Austrian competition in April 2026 and the European Championship in June served as the most important review points: it allowed us to test the first robot in real competition conditions and decide what to change in the future. We chose to build the mechanical and electronic base first, because without a stable platform we cannot reliably test the software. Once the platform was working, we focused on line following, then on the neural network and finally on the evacuation zone program.

Table 2. Project milestones, deadlines and responsible team members.

| Milestone                                              | Deadline       | Team member  |
|--------------------------------------------------------|----------------|--------------|
| Define overall robot concept and requirements          | September 2025 | Sven and Jan |
| 3D design of the first chassis                         | October 2025   | Jan          |
| Select microcontroller, sensors and motors             | October 2025   | Sven         |
| First version of the custom PCB                        | November 2025  | Sven         |
| Build first robot prototype                            | December 2025  | Sven and Jan |
| Basic line following with PID                          | December 2025  | Jan          |
| Green marker detection at intersections                | January 2026   | Jan          |
| First version of the victim handling arm               | January 2026   | Jan          |
| Collect and label first dataset for the neural network | January 2026   | Sven         |
| Train first neural network model for victim detection  | February 2026  | Sven         |

|                                                                                                        |                  |              |
|--------------------------------------------------------------------------------------------------------|------------------|--------------|
| National RoboCup Croatia 2026 (1 <sup>st</sup> place)                                                  | March 2026       | Sven and Jan |
| Identify weak points of robot v1 (weight, layout)                                                      | March 2026       | Sven and Jan |
| 3D design of the second, lighter chassis                                                               | March 2026       | Jan          |
| Second, smaller version of the PCB                                                                     | March 2026       | Sven         |
| Build second robot                                                                                     | March/April 2026 | Sven and Jan |
| Austrian RoboCup 2026 (2 <sup>nd</sup> place)                                                          | April 2026       | Sven and Jan |
| Improve line following especially on ramps                                                             | April 2026       | Jan          |
| Add the Hailo AI accelerator                                                                           | April 2026       | Sven         |
| Format NN to the Hailo format improving it in the process                                              | May 2026         | Sven         |
| European RoboCup 2026 (3 <sup>rd</sup> place)                                                          | June 2026        | Sven and Jan |
| Improve robustness and add lidars on both sides of the robot                                           | June 2026        | Sven         |
| Improve the NN and the evacuation zone program so it can work with obstructions in the evacuation zone | June 2026        | Sven         |
| Improve line following when there are obstructions on the track                                        | June 2026        | Jan          |
| RoboCup World Championship 2026                                                                        | July 2026        | Sven and Jan |

## b. Integration Plan

The robot is divided into two main subsystems that work together. The Teensy 4.1 handles all sensors, motor control and overall robot logic (state machine, PID, obstacle handling, victim handling arm). The Raspberry Pi handles the two cameras, image processing in OpenCV and the neural network used for victim detection in the evacuation zone. The two controllers communicate using the UART protocol. The Raspberry Pi sends messages with the position of the line, the type of intersection, and, in the evacuation zone, the position and type of detected victims. The Teensy uses this information together with the IMU and IR sensors to decide how to move and when. All components are connected either directly to the PCB or through short wiring, which gives us a clean and maintainable design.

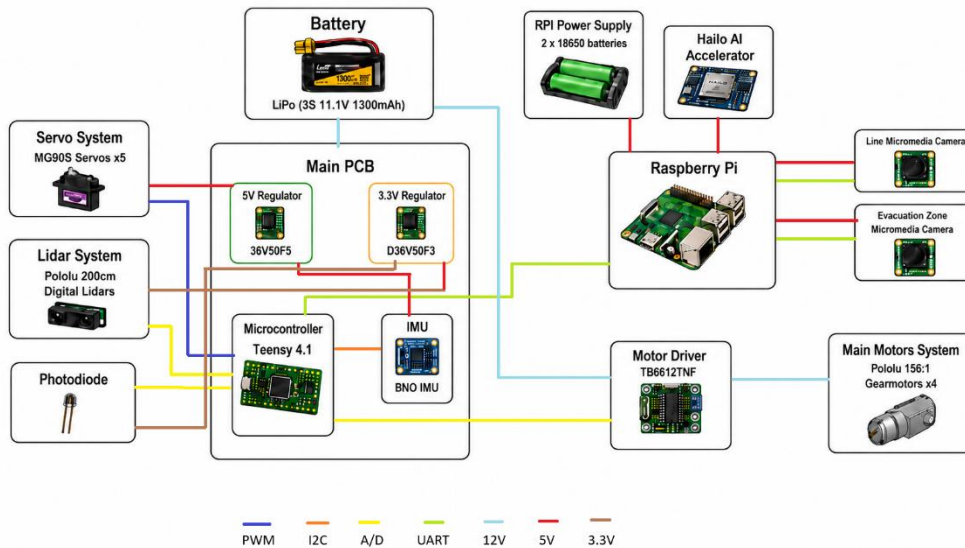
## 3. Hardware

The hardware of the robot was designed with three goals in mind: it must be light and compact, it must be reliable on the line and on ramps and it must be easy to repair and upgrade. We achieved this through a fully custom 3D printed chassis, a custom PCB and selected components.

### Main parts of the robot:

The robot consists of a 3D printed chassis, custom silicone-coated wheels, custom PCB, Teensy 4.1, Raspberry Pi 5, voltage regulators, motor driver, two cameras, IMU, IR distance sensors, four Pololu gearmotors, 11.1 V LiPo battery, four servos, start pushbutton and photodiodes.

Image 3 (scheme of connections)



## a. Mechanical Design and Manufacturing

### Main structure

The chassis is custom designed in Fusion 360 and Thinkercad and 3D printed. It was made to maintain a low centre of gravity, provide structural integrity, allow easy mounting of all components and make it possible to change parts quickly when something breaks during testing. The design keeps all major components securely mounted while still allowing easy access for maintenance. The robot is split into two main sections. The bottom section contains the motors, the battery and the line camera with its lighting diodes. The upper section contains the PCB, the Raspberry Pi, the wiring and the evacuation zone camera together with the other sensors. This layered structure improves accessibility and protects sensitive electronics from impacts during testing.

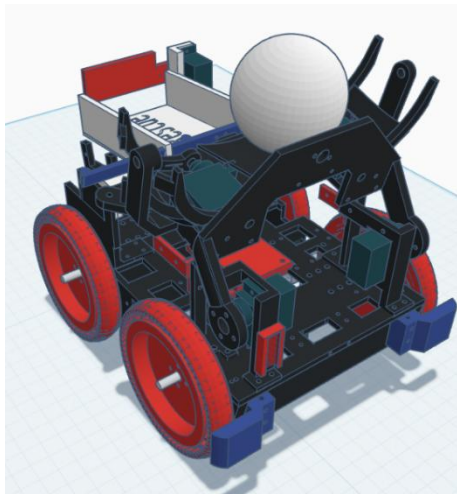


Image 4 (whole robot)

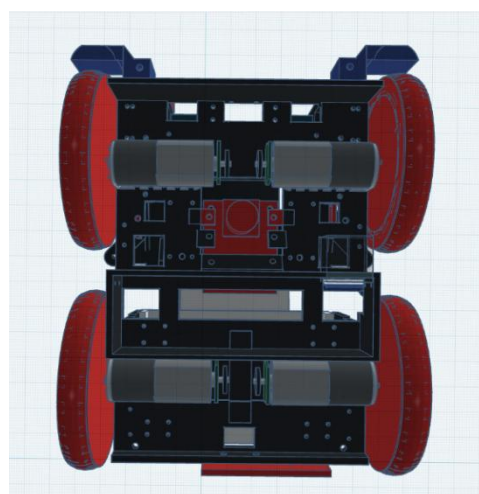


Image 5 (bottom section)



## Actuators and power train

For movement the robot uses four 156:1 Pololu metal gearmotors. These motors give us a good balance between torque and speed. The motors are driven by a motor driver mounted directly on the custom PCB.

## Victim handling system

The victim handling arm is mounted at the front of the robot and is one of the parts we redesigned most times. It uses 360° servos for grabbing the victims, 180° servos for raising and lowering the arm and a photodiode for confirming that a victim has actually been captured. The collection area on top of the robot holds the victims until the robot reaches the correct drop zone. Because the arm is the part that interacts physically with the victims, we tested it many times with the Rescue Line victims. We changed the shape of the grippers several times until the robot could pick up both live and dead victims reliably, even when they were placed close to walls.

## Custom debris removing tools

We expect a lot of debris like shredded papers, chopsticks and plastic. That is why we developed custom debris removing tools that can be mounted on the front of the robot to make line detection easier and to remove slipping effects caused by the debris. We developed two custom tools, one specialised for chopsticks and the other for shredded paper and plastic parts. The most important thing was that they still allowed the robot to pass bumps and ramps without any problem. We did that by integrating springs in our tool so it can bend but only for objects glued to the track.

## Custom silicone-coated wheels

To get the best possible grip we designed custom wheels with a 3D printed core and a silicone outer layer. The silicone provides high grip on the field surface, shock absorption when crossing intersections and bumps, better performance on ramps and the option to customize the wheel exactly for our robot. We tested several silicone hardness values before picking the one that gave the best compromise between grip and rolling resistance. This is one of the main reasons our second robot now passes ramps with bumps much better than the first.

## b. Electronic Design and Manufacturing

### Custom PCB

We designed our PCB in EasyEDA so that all components are integrated into a single compact system. The PCB reduces wiring complexity, improves reliability and minimizes the risk of short circuits during transport between competitions. Component placement was carefully planned so that signal routing is short and power and signal lines do not cross unnecessarily. We developed two versions of the PCB. The first version was too large and did not fit optimally inside the chassis, which made cable management difficult. The second version was redesigned to be significantly smaller and more efficient, which improved overall reliability and made the whole robot lighter.

Collection area

Arm for collecting victims

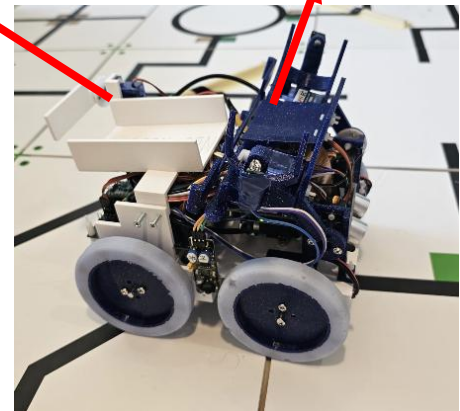


Image 6 (victim handling system)

Image 7 (PCB in EasyEDA software)

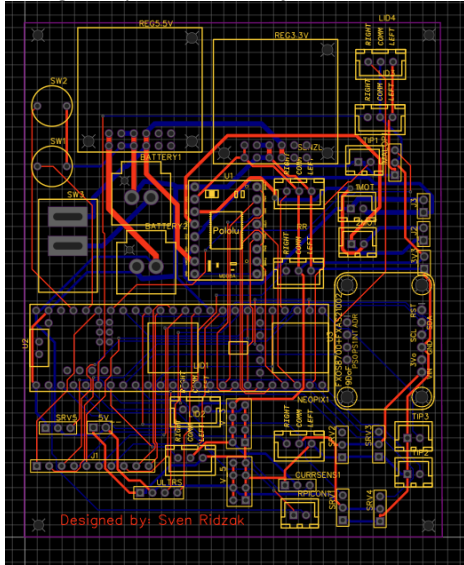
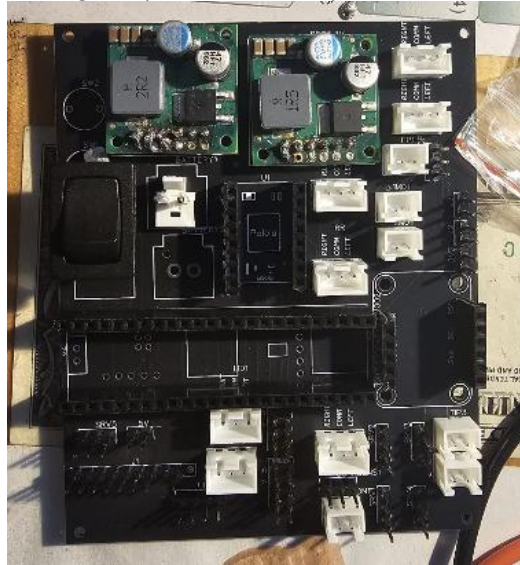


Image 8 (soldered PCB)



## Controller system

The Teensy 4.1 was selected as the main controller because of its high processing power, speed, Arduino compatibility and reliable performance. It is fast enough to run the PID control loop, the state machine and to read all sensors at the same time without missing any detections. The Raspberry Pi is used for all vision-related tasks and runs the neural network.

## Power system

The robot is powered by an 11.1 V LiPo battery. Voltage regulators provide stable 5 V and 3.3 V outputs for the logic, sensors and cameras. The battery is mounted in the lower section of the robot, close to the floor, which helps keep the centre of gravity low and improves stability on ramps and during sharp turns.

## Sensors used

The robot uses several sensors, each chosen for a specific task on the Rescue Line track.

- Line camera (Fishertechnik micromedia): used together with custom diode lighting for detecting the black line, intersections and the red line.
- Evacuation zone camera (PiCamera): wider angle camera used in the evacuation zone for the neural network.
- IMU: used for orientation, ramp detection and recovering from slips.
- IR distance sensors: used for distance measurement in the evacuation zone and for detecting the obstacle
- Photodiode: mounted on the arm to confirm that a victim has been captured.

## Hardware-accelerated neural network inference with Hailo AI HAT

Unlike our previous approach, where neural network inference was performed only in software on the Raspberry Pi CPU, we now use the Hailo AI HAT as a dedicated hardware accelerator for victim detection. The Hailo AI HAT allows the Raspberry Pi to offload neural network inference from the CPU to a specialized AI processor, significantly improving detection speed and reducing the processing load on the main system. After converting and optimizing our victim detection model for

the Hailo accelerator, we achieved fast inference to run victim detection in real time while the robot is moving inside the evacuation zone. This makes the system more responsive, especially when detecting victims from different angles or in partially obstructed positions.

### Custom diode lighting for the line camera

The light around the line camera was one of the biggest problems we had during early testing. Different rooms gave very different colours on the camera image, which made line detection and green dot detections unreliable. We solved this by building a custom controllable diode-based system around the line camera that is always on during the run. This gives a constant light source independent of the environment and makes the colour thresholds in OpenCV work reliably across all test fields.

## 4. Software

### a. General Software Architecture

The robot uses a software architecture split between two systems. The Teensy runs Arduino C++ code for motor control, PID and the IMU. The Raspberry Pi runs Python code using OpenCV for image processing and the Hailo accelerator for the neural network. The two systems communicate via UART, which gives us fast and reliable data transfer with very simple wiring. The software is organized in states, where each state corresponds to a specific phase of the run. The transitions between states are triggered by events from the sensors or the cameras (for example, detecting a silver line or finishing the victim search).

#### State based system

- State 0 – Idle: the robot is turned on but waiting for the start button.
- State 1 – Line following: the robot follows the black line, handles intersections and obstacles.
- State 2 – Evacuation zone navigation: the robot looks for victims using the neural network.
- State 3 – Victim rescue: the robot picks up victims and drops them in the correct area.
- State 4 – Evacuation zone exit: the robot searches for the exit.

#### Line following

The Raspberry Pi processes the line camera image and determines the line position and angle, the type of intersection and whether the robot is currently on a red line. This information is sent to the Teensy through UART. The Teensy then uses a PID controller to keep the robot centered on the line and to adjust speed during curves and ramps.

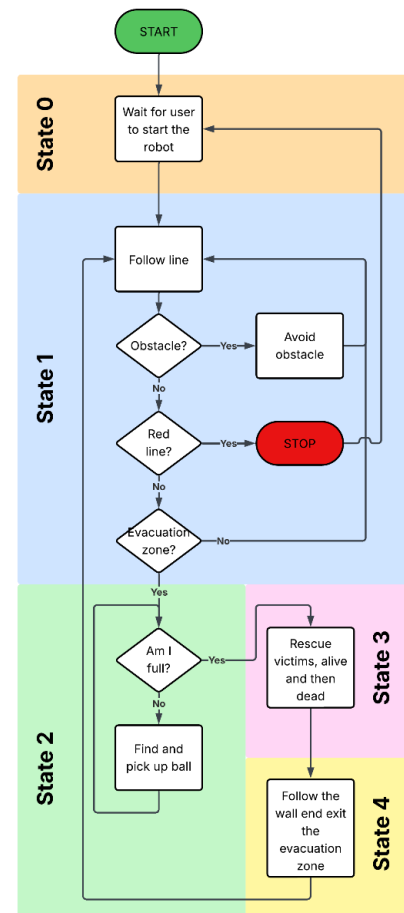


Image 9 (program states)



## Evacuation zone

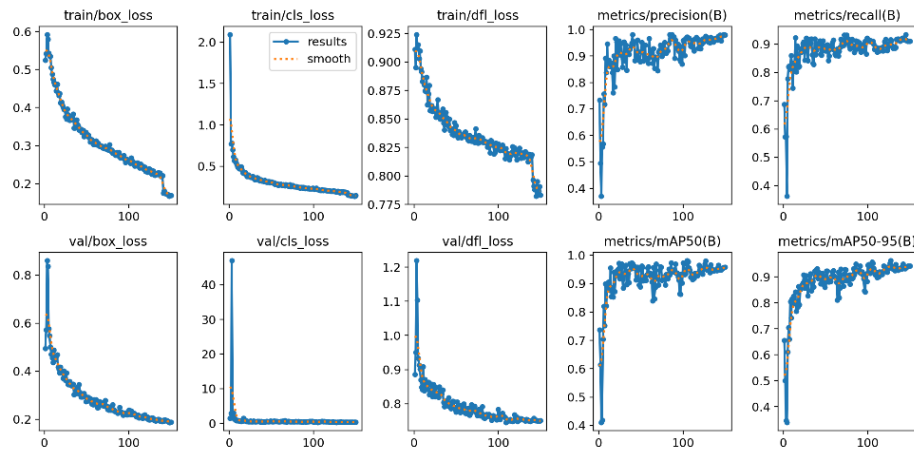
In the evacuation zone the robot switches to the second camera. A neural network detects the live victims, the dead victim and the collection areas. The robot navigates inside the zone, picks up the victims and finally searches for an exit.

After positioning itself in the centre of the evacuation zone, the robot starts spinning slowly and searches for victims with the neural network. After finding a victim, the robot drives towards it and picks it up with the arm. The robot first finds the two live victims and rescues them at the same time, and only after that it searches for the dead victim. Once all victims are delivered, the robot uses IR sensors to follow the wall and find the exit.

## Neural network

We trained the neural network using around 2000 images that we collected ourselves on different Rescue Line evacuation zones. The images were labelled in Roboflow and the model was trained on a local GPU using the Ultralytics framework. We chose a small model architecture so that it can run fast enough on the Raspberry Pi with the Hailo accelerator.

Image 10 (Loss and accuracy benchmarks)



## b. Innovative solutions

### Prioritising angle or position depending on the angle of the field

One of our main software innovations is our improved line following algorithm. Instead of using only the position of the line, our program calculates both the position and the angle of the line. This gives the robot more information about where the line is and how it is changing. The robot then changes how much it trusts each value depending on the situation. When the robot is driving on a normal flat line, it uses the line position and the line angle equally. This gives stable and smooth line following. When the robot is climbing a ramp, it focuses more on the line position, because staying centred is more important for keeping traction and avoiding slipping. When the robot is going down a ramp, it focuses more on the line angle, because the robot needs to react faster to direction changes and stay aligned with the line. This adaptive approach makes our line following more reliable on normal lines, ramps and difficult track sections.

### Stopping misidentifications in the evacuation zone

The second software innovation is our custom victim detection confirmation system. The neural network can sometimes detect a wrong object for only one frame, especially when the robot is

moving or when the victim is close to a wall. To reduce this problem, our program does not register a victim immediately. Instead, it checks whether the same object is detected for three consecutive frames. Only after that the robot confirms it as a real victim. This makes victim detection much more reliable. It reduces false detections and misidentifications, while still allowing the robot to move quickly inside the evacuation zone. Because of this, the robot can detect victims with high confidence without stopping for too long, which improves both speed and accuracy.

## 5. Performance evaluation

We tested every change we implemented on a Rescue Line field we built in our school robotics club from official-spec materials. After every major software update we always tested the extremes in real scenarios: lines with the longest legal gaps, intersections with both green markers on the same side, ramps with the maximum allowed angle and the evacuation zone with victims placed in corners and close to walls. The development of the robot was organized in iterations. After each iteration we ran a set of test runs and recorded how many of them the robot completed without any human help. This let us see which changes really improved performance and which only worked in a single situation.

### a. Line following performance

To evaluate the line following part of the robot we ran 20 attempts on each test scenario, both with the first robot (v1) and with the second, improved robot (v2). The robot is counted as successful only if it solves the whole scenario without any manual intervention. The results are shown in the table below.

Table 3. Line following success rate, robot v1 vs robot v2.

| Test scenario                               | Robot v1 success rate | Robot v2 success rate |
|---------------------------------------------|-----------------------|-----------------------|
| Simple straight line and curves             | 95%                   | 100%                  |
| Line with gaps                              | 70%                   | 95%                   |
| Intersection with green marker (left/right) | 75%                   | 100%                  |
| Ramps with bumps and green marker           | 30%                   | 90%                   |
| Obstacle on the line                        | 60%                   | 95%                   |
| Red line detection and stop                 | 70%                   | 100%                  |

The biggest improvements between v1 and v2 came from three changes: the lower weight of the second chassis (which made the wheels grip better on ramps with bumps), the custom diode lighting (which made the line and the red line easier to detect in different rooms) and the improved obstacle handling routine that uses IR sensors. After these changes the robot finishes the line part of the track reliably in almost every test run.

### b. Evacuation zone and neural network performance

To evaluate victim detection we collected an extra test set of 100 images that were not used during training. The table below compares the first model we trained (using NCNN specifically made software for optimising the NN model on the Raspberry Pi CPU) with the final model using Hailo accelerator.

Table 4. Neural network performance before and after NCNN optimization.

| Test scenario                                | First NN model (NCNN) | Final NN model (Hailo) |
|----------------------------------------------|-----------------------|------------------------|
| Detect live victim in the centre of the zone | 78 %                  | 96 %                   |
| Detect victim close to the robot             | 65 %                  | 90 %                   |
| Detect victim close to a wall                | 50 %                  | 95 %                   |
| Detect drop area                             | 80 %                  | 98 %                   |

The success rate improved significantly after we retrained the model with extra images collected from real Rescue Line fields and added the Hailo accelerator, especially for victims close to walls, which was the hardest case in our earlier tests.

### c. Impact on development

The results of these tests directly drove our development. For example, the low success rate of the first robot on obstacles is the reason we added a better IR sensor in robot v2 that can detect all kinds of obstacles. The low success rate of the first neural network on victims close to walls is the reason we collected more training images from that specific situation and retrained the model.

## 6. Conclusion

Over the past months we put a lot of time, effort and creativity into building this robot, and we are proud of what we have achieved. The result reflects our hard work and dedication, and while we still see room for improvement, that only motivates us for the next competitions. Along the way we learned a lot about mechanical design, PCB design, computer vision and neural networks and we developed many new ideas we are eager to explore further. This robot represents a significant improvement over our previous designs. Through iterative development, careful hardware design and a robust software architecture, we created a reliable and efficient robot. Extensive testing allowed us to identify weak points and continuously improve performance. The result is a robot capable of handling all elements of the Rescue Line field. We are looking forward to sharing our experience and exchanging insights with other teams at RoboCupJunior 2026.

## References

- Arduino – <https://www.arduino.cc/>
- Teensy 4.1 – <https://www.pjrc.com/store/teensy41.html>
- Raspberry Pi – <https://www.raspberrypi.com/>
- Pololu – <https://www.pololu.com/>
- Adafruit – <https://www.adafruit.com/>
- PCBWay – <https://www.pcbway.com/>
- EasyEDA – <https://easyeda.com/>
- OpenCV – <https://opencv.org/>
- Ultralytics – <https://www.ultralytics.com/>
- Hailo – <https://hailo.ai/products/hailo/>
- Roboflow – <https://roboflow.com/>